DocBrain AI Platform

End-to-End Technical Specification & Pipeline Architecture (A to Z)

Document ID: DOCBRAIN-PIPELINE-SPEC-2026	**Release Version:** 1.0.0-RELEASE
Author: Core Architecture & Systems Engineering Group	**Classification:** Production Technical Specification
Target Audience: Architects, Developers, DevOps, Security Leads	**Effective Date:** August 9, 2026

EXECUTIVE SUMMARY: DocBrain AI is an enterprise multi-tenant document intelligence microservices platform. Built on Next.js 15, Node.js Express, Redis Pub/Sub, Python FastAPI, LangGraph, ChromaDB, and MongoDB Atlas, it provides asynchronous document processing, anti-hallucinated hybrid RAG Q&A;, executive summaries, concept mind maps, and text-to-speech audio podcasts. This specification provides a complete A-to-Z breakdown of every architecture component and execution pipeline flow.

System Core Metrics & SLA Targets:

Service Layer	Core Technology	SLA / Operational Guarantee
Frontend Client	Next.js 15 App Router, React 18, Zustand	< 100ms Page Load, Global CDN
API Gateway	Node.js Express, TypeScript, Zod DTOs	< 150ms Gateway Response (p95)
Event Broker	Redis Pub/Sub Event Channels	Zero Main Thread API Blocking
AI Microservice	Python FastAPI, LangGraph, ChromaDB	> 99% Retrieval Accuracy (RRF)
Multi-Modal Output	Groq, Gemini, gTTS Audio Engine	Q&A;, Summary, Mind Map, Audio MP3

Chapter 1: Architecture Blueprint & Monorepo Design

DocBrain AI enforces strict microservice decoupling. Direct synchronous HTTP requests between the Node.js API Gateway and the Python AI processing engine for heavy tasks are strictly forbidden. All heavy workloads (PDF parsing, text chunking, vector indexing, and RAG execution) are dispatched asynchronously through Redis Pub/Sub event queues.

1.1 Monorepo Workspace Directory Blueprint

```
pdf-kb-ai-chatbot/ ■■■ apps/ ■ ■■■ frontend/ # Next.js 15 App Router Frontend Client ■ ■ ■■■ src/app/ # App Router Layouts, Error Boundaries, Pages ■ ■ ■■■ src/components/ # UI Primitives & Providers ■ ■ ■■■ src/features/ # Auth, Documents, Chat, Audio, MindMap modules ■ ■ ■■■ src/store/ # Zustand State Management Stores ■ ■■■ backend/ # Node.js Express TypeScript API Gateway ■ ■ ■■■ src/config/ # Environment, Database & Swagger Specs ■ ■ ■■■ src/features/ # Auth, Document, Chat Controllers & Routes ■ ■ ■■■ src/middlewares/ # Auth, RBAC, Multer File Upload, Zod Validation ■ ■ ■■■ src/repositories/ # Data Access Layer Repositories ■ ■ ■■■ src/redis/ # Redis Pub/Sub Event Dispatcher ■ ■■■ ai-service/ # Python FastAPI AI Microservice Engine ■ ■■■ app/api/ # FastAPI Endpoints (/ingest, /query, /health) ■ ■■■ app/db/ # ChromaDB Vector Store Client Manager ■ ■■■ app/redis_pubsub/ # Async Redis Subscriber & Retry Loops ■ ■■■ app/services/ # PDFProcessor, HybridRetriever, LangGraph RAG ■■■ packages/ ■ ■■■ shared/ # Shared Interfaces & Redis Event Contracts ■■■ docker-compose.yml # Complete Multi-Container Orchestration
```

1.2 Microservice Isolation & Domain Responsibilities

Each microservice in the monorepo has distinct, non-overlapping architectural duties:

- **Frontend Service (Next.js 15):** Handles stateful UI rendering, Markdown parsing, audio player controls, mind map visualization, and JWT token management.
- **API Gateway (Node.js Express):** Enforces user authentication, password hashing, RBAC file ownership, multipart file upload validation, database queries, and Redis event dispatching.
- **AI Processing Engine (Python FastAPI):** Operates background workers for PyPDF text extraction, character chunking, ChromaDB vector indexing, BM25 sparse search, LangGraph stateful execution, Groq/Gemini LLM inference, and gTTS audio synthesis.

Chapter 2: Pipeline Flow 1 — Asynchronous PDF Ingestion (A to Z)

The PDF Document Ingestion Pipeline processes unstructured raw PDF files into structured, indexed vector embeddings. To guarantee zero main-thread API freezing on the web backend, the entire ingestion flow is executed asynchronously.

2.1 Step-by-Step Ingestion Flow Execution (A to Z)

Step 1: File Submission: User selects a PDF document and submits it via Next.js multipart form data.

Step 2: Gateway Security: Express Gateway validates JWT token, checks Multer MIME type (application/pdf), and verifies file size (max 25MB).

Step 3: Storage & Metadata: Gateway saves file to disk with cryptographically unique filename (-.pdf) and creates MongoDB Document record with status: PENDING.

Step 4: Redis Event Dispatch: Gateway publishes pdf.ingest event payload (documentId, userId, filePath) to Redis Pub/Sub and returns HTTP 202 Accepted to user in < 50ms.

Step 5: Background Queue Pickup: Python AI Microservice Redis Subscriber consumes event and updates status to PROCESSING.

Step 6: PyPDF Text Extraction: Python PDFProcessor loads PDF binary, extracts plain text per page, and tracks page numbers.

Step 7: Context Chunking: LangChain RecursiveCharacterTextSplitter splits text into chunks (chunk_size = 1000, chunk_overlap = 200).

Step 8: Vector Embedding: SentenceTransformers generates 384-dimensional dense vector embeddings for each chunk.

Step 9: ChromaDB Indexing: Vectors and metadata (documentId, pageNumber, textSnippet) indexed into ChromaDB persistent collection.

Step 10: Status Push: Python worker publishes pdf.ingest.status (COMPLETED, totalChunks) to Redis Pub/Sub, updating MongoDB and client UI.

2.2 Ingestion Event JSON Payload Schema

```
// Redis Channel: pdf.ingest { "eventId": "evt_ingest_992813019", "documentId":
"66b5f4a1e9b2c3d4e5f6a7b8", "userId": "66b5f4a1e9b2c3d4e5f6a001", "filePath":
"/uploads/documents/1723165000-technical_manual.pdf", "fileName": "technical_manual.pdf", "timestamp":
"2026-08-09T03:55:00.000Z" }
```

Chapter 3: Pipeline Flow 2 — Hybrid Search & Vector Retrieval (A to Z)

Standard vector search relies solely on dense embeddings (Cosine Distance), which fails on exact alphanumeric queries (e.g., searching for 'Error Code 403-X' or 'Part #9921'). DocBrain AI implements Hybrid Search, combining Dense Semantic Vectors and Sparse Keyword Search (BM25) using Reciprocal Rank Fusion (RRF).

3.1 Reciprocal Rank Fusion (RRF) Formulation

Given a query q, the hybrid retriever executes parallel searches in ChromaDB (Dense) and BM25 (Sparse). The Reciprocal Rank Fusion score RRF(d) for document chunk d across rankings is calculated as:

```
RRF_Score(d ∈ D) = ∑ [ 1 / ( k + Rank_dense(d) ) ] + [ 1 / ( k + Rank_sparse(d) ) ] Where: • k = 60
(Standard smoothing constant preventing top-rank bias) • Rank_dense(d) = Ordinal rank in ChromaDB
vector similarity search • Rank_sparse(d) = Ordinal rank in BM25 exact keyword match search
```

3.2 Step-by-Step Retrieval Execution Pipeline

1. **Query Received:** User enters question in Chat UI. Express Gateway dispatches chat.query event via Redis Pub/Sub.
2. **Parallel Retrieval:** Python HybridRetriever fires ChromaDB vector query and BM25 text query simultaneously.
3. **RRF Score Merging:** Reciprocal Rank Fusion merges rankings and selects Top-5 highest scoring document chunks.
4. **Tenant Security Filter:** Filters chunks by documentId and userId permissions to prevent cross-tenant data leaks.
5. **Context Construction:** Formats retrieved chunks into structured context text with document citations and page numbers.

Chapter 4: Pipeline Flow 3 — LangGraph & Anti-Grounding Execution

DocBrain AI utilizes LangGraph StateGraph orchestration to execute stateful RAG query workflows. The graph enforces strict anti-hallucination prompting and automatically mines 3 grounded follow-up questions.

4.1 LangGraph StateGraph Architecture

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ State: RAGQueryState ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Node: hybrid_retrieve ■ ➔ Hybrid Dense + BM25 RRF Search
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Node: generate_answer ■
➔ Strict Context Prompt + Groq/Gemini ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■Node: generate_followups ■ ➔ Auto-mines 3 grounded questions
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

4.2 Grounded System Prompt Template

```
SYSTEM PROMPT: You are DocBrain AI, an enterprise technical document assistant. You must answer the
user's question STRICTLY based on the provided context below. If the answer cannot be directly derived
from the context, state clearly: "I cannot find sufficient information in the uploaded document to
answer this question." Do NOT invent, assume, or extrapolate facts outside the context. CONTEXT:
{retrieved_context_chunks} USER QUESTION: {user_query}
```

Chapter 5: Pipeline Flow 4 — Multi-Modal Information Transformation

DocBrain AI extends basic text chat by offering four distinct multi-modal information transformation services: Conversational Q&A;, Executive Bullet Summarization, Interactive Concept Mind Maps, and Text-to-Speech Audio Overviews.

5.1 Executive Document Summarization Pipeline

1. Python SummaryService extracts all text chunks for documentId.
2. Executes Map-Reduce summarization chain using Groq LPU / Gemini Flash.
3. Formats summary into key bullet insights and stores result in MongoDB Document record.

5.2 Interactive Mind Map JSON Tree Pipeline

1. Python MindMapService extracts document hierarchy and key concepts.
2. Generates structured JSON node tree (root, main topics, sub-nodes).
3. Next.js Frontend renders dynamic interactive SVG concept tree.

5.3 Text-to-Speech (gTTS) Audio Overview Pipeline

1. Python AudioService converts executive summary text into 24kHz audio stream.
2. Google Text-to-Speech (gTTS) synthesizes MP3 file saved to ./uploads/audio/.mp3.
3. Frontend renders HTML5 Audio Player with custom playback controls.

Chapter 6: Enterprise Security, RBAC & API Governance

6.1 Authentication & Token Lifecycle

• **Password Hashing:** Passwords hashed using bcrypt with salt factor of 10.

• **JWT Bearer Tokens:** Authenticated routes require Authorization: Bearer header.

• **Public Share Tokens:** Cryptographically signed tokens (/share/:token) allow guest viewing without exposing private user accounts.

6.2 Role-Based Access Control (RBAC) Matrix

Resource / Action	Unauthenticated Guest	Standard User (USER)	Admin (ADMIN)
Register / Login	■ Allowed	■ Allowed	■ Allowed
Upload PDF Document	■ Denied	■ Owned Workspace	■ Allowed
Delete Document	■ Denied	■ Owned Document Only	■ Any Document
Query RAG Chat	■ Denied	■ Owned Workspace	■ Allowed
Public Share Link View	■ Allowed (/share/:token)	■ Allowed	■ Allowed

Chapter 7: Database & Persistence Layer Specifications

7.1 MongoDB Atlas Data Schemas (Mongoose)

```
// Document Model (documents) const DocumentSchema = new Schema({ userId: { type:
Schema.Types.ObjectId, ref: 'User', required: true }, fileName: { type: String, required: true },
filePath: { type: String, required: true }, fileSize: { type: Number, required: true }, status: { type:
String, enum: ['PENDING', 'PROCESSING', 'COMPLETED', 'FAILED'], default: 'PENDING' }, totalChunks: {
type: Number, default: 0 }, summary: { type: String }, mindMapJson: { type: Object }, audioPath: {
type: String }, shareToken: { type: String, unique: true }, }, { timestamps: true });
```

7.2 ChromaDB Vector Database Schema

ChromaDB stores document chunk embeddings in a persistent collection with payload metadata:

- **Collection Name:** pdf_knowledge_base
- **Embedding Dimension:** 384 (SentenceTransformers all-MiniLM-L6-v2)
- **Distance Metric:** Cosine Similarity (1 - Cosine Distance)
- **Metadata Fields:** documentId, userId, pageNumber, chunkIndex, textSnippet

Chapter 8: DevOps, Containerization & Cloud Deployment

8.1 Docker Compose Orchestration (docker-compose.yml)

```
version: '3.8' services: redis: image: redis:7-alpine ports: ['6379:6379'] chromadb: image:
chromadb/chroma:latest ports: ['8000:8000'] backend: build: { context: ., dockerfile:
apps/backend/Dockerfile } ports: ['5000:5000'] environment: - REDIS_URL=redis://redis:6379 -
MONGO_URI=mongodb://mongo:27017/pdf_kb ai-service: build: { context: ./apps/ai-service, dockerfile:
Dockerfile } ports: ['8001:8001'] environment: - REDIS_URL=redis://redis:6379
```

8.2 Cloud Hosting Stack Breakdown

- **Frontend Client:** Vercel Cloud Platform (Global Edge CDN).
- **API Gateway & AI Microservice:** Render Web Services (Dockerized Containers).
- **Database Layer:** MongoDB Atlas Cloud Managed M0 Cluster.
- **Event Broker:** Upstash Serverless Redis Pub/Sub.

Chapter 9: API Endpoint Reference Catalogue

Method	Endpoint Path	Auth Required	Description
POST	/api/v1/auth/register	No	Registers new user account
POST	/api/v1/auth/login	No	Authenticates user & returns JWT
POST	/api/v1/documents/upload	JWT Bearer	Uploads PDF & dispatches Redis event
GET	/api/v1/documents	JWT Bearer	Lists owned documents
POST	/api/v1/chat/query	JWT Bearer	Executes RAG chat query
GET	/api/v1/documents/:id/summary	JWT Bearer	Fetches executive summary
GET	/api/v1/documents/:id/mindmap	JWT Bearer	Fetches mind map JSON tree
GET	/api/v1/documents/:id/audio	JWT Bearer	Fetches TTS audio MP3 link
GET	/health	No	Backend API Health check
GET	/health/redis	No	Redis Pub/Sub Health check

Chapter 10: Technical Appendix & Product Roadmap

10.1 Product Development Roadmap

- **Phase 1 (Current v1.0):** PDF RAG, Summaries, Mind Maps, Audio Overview, Decoupled Redis Architecture.
- **Phase 2 (Target Q4 2026):** Multi-format ingestion (.docx, .pptx, .xlsx, OCR scanned PDFs).
- **Phase 3 (Target Q2 2027):** SAML/Okta Enterprise SSO & Fine-grained team permissions.
- **Phase 4 (Target Q4 2027):** Distributed Qdrant vector DB clusters for billion-vector scaling.

10.2 Technical Specification Approval Sign-Off

Role	Name	Signature	Date
Chief Technology Officer	Architecture Review Board	APPROVED	August 9, 2026
VP of Engineering	DocBrain Systems Engineering	APPROVED	August 9, 2026
Lead DevOps Engineer	Infrastructure Operations	APPROVED	August 9, 2026